BLOCKCHAIN TECHNOLOGY

Project Report

Developing a Blockchain-Based Solution for Women Empowerment in Semi-Urban Rwanda

Application Area: Supply Chain Management

Platform: Ethereum Sepolia Testnet

2025 / 2026 Academic Year

1. Problem Identification and Alignment with Mission

1.1 Background — Women-Led Businesses in Semi-Urban Rwanda

Semi-urban Rwanda encompasses rapidly growing towns and peri-urban zones such as Musanze, Huye, Rubavu, and Nyagatare. Women in these areas form the backbone of local economies through small and medium enterprises (SMEs) focused on agriculture, handcrafts, food processing, and textiles. Despite their economic significance, these women entrepreneurs face severe structural barriers that prevent them from scaling their businesses and accessing broader markets.

1.2 The Specific Problem

The core problem identified is the lack of a transparent, tamper-proof, and accessible supply chain tracking system for women-led businesses in semi-urban Rwanda. This manifests in three specific, interconnected challenges:

- Counterfeit and Imitation Products: Authentic handmade products from women cooperatives are frequently undercut by cheaper imitations in local and regional markets. Without a verifiable proof of origin, buyers cannot distinguish genuine products from counterfeits, forcing authentic producers to compete on price rather than quality.
- Lack of Market Trust and Transparency: Buyers — particularly urban consumers and international NGO buyers — have no way to verify where a product came from, who made it, what materials were used, and how it travelled through the supply chain. This information asymmetry works against the women producers who cannot command fair prices.
- Financial Exclusion from Formal Markets: Without documented proof of production history and consistent sales records, women-led businesses cannot access formal credit, government procurement programs, or export opportunities. Traditional record-keeping is paper-based, easily lost, and unverifiable by third parties.

1.3 Evidence of the Problem

According to the Rwanda Development Board and USAID Rwanda studies, over 68% of women-led SMEs in semi-urban areas report losing market share to counterfeit products. Furthermore, fewer than 12% of women cooperatives maintain auditable supply chain records. The IFC estimates that Rwandan women entrepreneurs face a financing gap of over $700 million USD annually, partly due to inability to demonstrate consistent business activity.

1.4 Alignment with Personal Mission

My personal mission is to use technology as an equalizer — to build systems that give the most marginalized communities access to tools previously reserved for large corporations. The women of semi-urban Rwanda are highly skilled, hardworking, and entrepreneurial, yet they are excluded from digital economies simply because they lack access to the right infrastructure.

Solving this supply chain transparency problem directly aligns with this mission because it:

- Democratizes trust: Any buyer — whether in Kigali, Nairobi, or Paris — can verify a product's origin and journey with a simple blockchain lookup, without needing to trust any single institution.
- Removes intermediaries: The smart contract transfers payment directly from buyer to producer in real time, eliminating exploitative middlemen who currently capture 30–50% of the product value.
- Creates a digital footprint: Each registered product and completed transaction builds a verifiable business history on the blockchain, which can be presented to microfinance institutions as proof of economic activity.
- Requires no bank account: Participation requires only a smartphone with MetaMask, making it accessible to the unbanked population that dominates semi-urban Rwanda.

2.1 Proposed Solution Overview

The proposed solution is the WomenSupplyChain platform — a decentralized application (dApp) built on the Ethereum blockchain that allows women-led businesses in semi-urban Rwanda to register their businesses, list their products, track each product through its supply chain journey, and receive direct payments from buyers without intermediaries.

The platform operates entirely on-chain through a Solidity smart contract deployed on the Ethereum Sepolia Testnet. There is no central database, no central server, and no administrator who can alter records. Once data is recorded on the blockchain, it is immutable and publicly verifiable.

2.2 How Blockchain Technology is Utilized

Blockchain technology is leveraged in the following specific ways within this solution:

- Immutability: Once a business is registered, a product is listed, or a supply chain stage is recorded, that data cannot be altered or deleted. This guarantees authenticity and prevents fraud.
- Decentralization: No single entity controls the platform. The smart contract executes autonomously according to its coded rules, removing the need for a trusted intermediary.

- Smart Contract Automation: Payment is automatically transferred from buyer to seller the moment a purchase is confirmed, with no manual processing, delays, or bank transfer fees.
- Transparency and Public Verifiability: Every transaction and supply chain stage emits a blockchain event that can be queried by anyone using a block explorer like Etherscan, providing a fully auditable history.
- Wallet-Based Identity: Each business owner is identified by their Ethereum wallet address, which serves as a decentralized identity requiring no government ID or bank account to create.

2.3 Key Features and Functionalities

Feature 1: Business Registration

Women entrepreneurs register their business on the blockchain by submitting their business name and semi-urban district location. The registration is tied to their wallet address and stored permanently on-chain. No duplicate registrations are possible for the same wallet address.

Feature 2: Product Registration

Registered businesses can list products with a name, category (e.g., Crafts, Agriculture, Food Processing), and price in ETH. Each product receives a unique, auto-incrementing ID and is marked as available for purchase by default.

Feature 3: Supply Chain Stage Tracking

Product owners can add supply chain stages to any of their products, recording the stage name (e.g., Harvested, Processed, Quality Checked, Shipped), the location where the stage occurred, and any additional notes. Each stage is timestamped by the blockchain. Buyers can query the full history of any product before purchasing.

eature 4: Direct Purchase with ETH Transfer

Buyers can purchase any available product by sending the exact ETH amount. The smart contract immediately transfers the payment to the producer's wallet address with no intermediary fees. The product is marked as unavailable after purchase, preventing double-selling.

Feature 5: Availability Management

Product owners can toggle the availability status of their products, allowing them to temporarily suspend listings without deleting the product record.

2.4 Role of the Smart Contract

The smart contract (WomenSupplyChain.sol) is the central engine of the entire solution. It performs all of the following functions autonomously:

- Acts as the persistent database: All business, product, supply chain, and purchase records are stored in contract state variables (mappings and structs).
- Enforces business rules: Modifiers ensure only registered businesses can list products, only product owners can add supply chain stages, and buyers cannot purchase their own products.

- Executes payments: The payable purchaseProduct function receives ETH from buyers and forwards it directly to producers using a low-level call, ensuring atomic and trustless transfers.
- Emits audit events: Every significant action emits a blockchain event (BusinessRegistered, ProductRegistered, StageAdded, ProductPurchased) that creates a permanent, queryable audit trail.
- Provides public read access: View functions allow anyone to read business details, product information, supply chain stages, and purchase records without paying gas fees.

3. Smart Contract Development

3.1 Development Environment

The smart contract was developed using the following tools and technologies:

- Language: Solidity 0.8.19
- Development Framework: Hardhat 2.22.0
- Runtime: Node.js with npm
- IDE: Visual Studio Code with the Solidity extension by Juan Blanco
- Testing Library: Chai (assertion library) + Hardhat Ethers plugin
- Wallet: MetaMask browser extension
- Testnet: Ethereum Sepolia
- RPC Provider: Alchemy
- Block Explorer: Etherscan (Sepolia)

3.2 Smart Contract Architecture

The contract is structured around four core data structures stored using Solidity mappings:

Struct	Key Fields	Purpose
Business	owner, name, location, isRegistered, registeredAt	Stores women-led business profile tied to wallet address
Product	id, name, category, producer, price, isAvailable, createdAt, stageCount	Stores product listing with pricing and availability
SupplyStage	stageName, location, notes, recordedBy, timestamp	Stores one step in the product journey through the supply chain
Purchase	productId, buyer, seller, price, timestamp	Stores a completed purchase record as permanent proof of sale

3.3 Key Smart Contract Functions

The contract exposes the following public functions. Each function is fully commented in the source code:

```
// Business Management
registerBusiness(string _name, string _location) external
getBusiness(address _owner) external view returns (...)

// Product Management
registerProduct(string _name, string _category, uint256 _price) external
getProduct(uint256 _productId) external view returns (...)
getBusinessProducts(address _owner) external view returns (uint256[])
setProductAvailability(uint256 _productId, bool _isAvailable) external

// Supply Chain Tracking
addSupplyStage(uint256 _productId, string _stageName, string _location, string _notes)
external
getSupplyStage(uint256 _productId, uint256 _stageIndex) external view returns (...)
getStageCount(uint256 _productId) external view returns (uint256)

// Purchase & Payment
purchaseProduct(uint256 _productId) external payable
getPurchase(uint256 _purchaseId) external view returns (...)

// Utility
getTotalProducts() external view returns (uint256)
getTotalPurchases() external view returns (uint256)
```

3.4 Access Control Modifiers

Three custom modifiers enforce access control rules throughout the contract:

```
modifier onlyRegisteredBusiness() {
  require(businesses[msg.sender].isRegistered,
    "You must register your business first");
  _;
}

modifier onlyProductOwner(uint256 _productId) {
  require(products[_productId].producer == msg.sender,
    "Only the product owner can do this");
  _;
}

modifier productExists(uint256 _productId) {
  require(_productId > 0 && _productId <= productCounter,
    "Product does not exist");
  _;
}
```

3.5 Payment Transfer Logic

The purchase function implements a secure ETH transfer pattern using the low-level call method, which is the recommended approach per Solidity best practices and avoids reentrancy vulnerabilities when combined with the checks-effects-interactions pattern:

```
function purchaseProduct(uint256 _productId) external payable {
  Product storage p = products[_productId];
  require(p.isAvailable, "Product is not available for sale");
  require(msg.sender != p.producer, "Producer cannot buy own product");
```

```
    require(msg.value >= p.price, "Insufficient payment");

    // EFFECTS: update state before external call
    p.isAvailable = false;
    purchaseCounter++;
    purchases[purchaseCounter] = Purchase({...});

    // INTERACTIONS: transfer ETH to producer
    (bool sent, ) = payable(p.producer).call{value: msg.value}("");
    require(sent, "Payment transfer failed");

    emit ProductPurchased(_productId, msg.sender, p.producer, msg.value);
}
```

4. Implementation

4.1 Project Setup — Step by Step

The following is a complete, reproducible guide to setting up and deploying the WomenSupplyChain smart contract from scratch on Ubuntu/WSL with VS Code.

Step 1: Create Project Directory

```
mkdir women-supply-chain
cd women-supply-chain
code .
```

Step 2: Initialize Node.js Project

```
npm init -y
```

Step 3: Install Hardhat v2

```
npm install --save-dev hardhat@2.22.0
npm install --save-dev @nomicfoundation/hardhat-toolbox@5
npm install dotenv
npx hardhat init
# Select: Create a JavaScript project
# Press Enter for all prompts
```

Step 4: Create Project Files

```
# Create scripts folder if missing
mkdir scripts

# Remove Hardhat sample files
rm contracts/Lock.sol
rm test/Lock.js

# Create my files in VS Code:
# contracts/WomenSupplyChain.sol
# scripts/deploy.js
# test/WomenSupplyChain.test.js
# Replace hardhat.config.js
# Create .env file
```

Step 5: Configure hardhat.config.js

```
require('@nomicfoundation/hardhat-toolbox');
require('dotenv').config();

module.exports = {
  solidity: '0.8.19',
  networks: {
    hardhat: {},
    sepolia: {
      url: process.env.SEPOLIA_RPC_URL || '',
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
    },
  },
  etherscan: {
    apiKey: process.env.ETHERSCAN_API_KEY || '',
  },
};
```

Step 6: Configure .env File

```
SEPOLIA_RPC_URL=https://eth-sepolia.g.alchemy.com/v2/MY_ALCHEMY_KEY
PRIVATE_KEY=MY_TEST_WALLET_PRIVATE_KEY
ETHERSCAN_API_KEY=MY_ETHERSCAN_API_KEY
```

Step 7: Get Sepolia Testnet ETH

```
# Visit the Google faucet:
# https://cloud.google.com/application/web3/faucet/ethereum/sepolia
# Paste your MetaMask test wallet address
# Receive 0.05 Sepolia ETH for free
```

Step 8: Compile the Contract

```
npx hardhat compile
# Expected output:
# Compiled 1 Solidity file successfully (evm target: paris).
```

Step 9: Run Tests

```
npx hardhat test
# Expected output:
# 23 passing (52s)
```

Step 10: Deploy to Sepolia Testnet

```
npx hardhat run scripts/deploy.js --network sepolia

# Expected output:
# Deploying WomenSupplyChain contract...
# Deploying with account: 0xfacaA0D21Df37D9f1D1bA28020631881ab027C42
# Account balance: 0.05 ETH
# WomenSupplyChain deployed to: 0xAaB5646895d0F81dc45C0D6D8c3f9807db9F96D9
# View on Etherscan: https://sepolia.etherscan.io/address/0xAaB5...
```

4.2 Successful Deployment Evidence

Detail	Value
Deployment Status	SUCCESS — Contract live on Sepolia Testnet
Contract Address	0xAaB5646895d0F81dc45C0D6D8c3f9807db9F96D9
Deployer Address	0xfacaA0D21Df37D9f1D1bA28020631881ab027C42
Network	Ethereum Sepolia Testnet (Chain ID: 11155111)
Compiler	Solidity 0.8.19, EVM target: Paris
Gas Used	Approximately 0.0003 Sepolia ETH
Block Explorer	https://sepolia.etherscan.io/address/0xAaB5...
Test Results	23 / 23 test cases PASSING

5. Validation and Testing

5.1 Testing Approach

All smart contract functions were tested using the Hardhat testing framework with the Chai assertion library. Tests were organized into 5 test suites covering all major contract features. Each test case is designed to validate both the happy path (expected successful behavior) and the sad path (expected rejections and error conditions).

The test suite was run on the local Hardhat network which simulates the Ethereum blockchain environment, providing deterministic and fast test execution without gas costs. All 23 test cases passed successfully.

5.2 Test Results Summary

ID	Test Description	Expected Outcome	Actual Result	Status
TC-01	Register a new business successfully	Business stored on-chain with correct name, location, isRegistered=true	Business registered, fields verified	**PASS**
TC-02	Reject duplicate business registration	Revert: 'Business already registered'	Transaction reverted with correct message	**PASS**
TC-03	Reject empty business name	Revert: 'Business name cannot be empty'	Transaction reverted with correct message	**PASS**
TC-04	Reject empty location	Revert: 'Location cannot be empty'	Transaction reverted with correct message	**PASS**
TC-05	Emit BusinessRegistered event	Event emitted with correct address, name, location	Event verified with correct arguments	**PASS**
TC-06	Register a product successfully	Product stored with id=1, correct fields, isAvailable=true	Product registered and all fields verified	**PASS**
TC-07	Reject product from unregistered business	Revert: 'You must register your business first'	Transaction reverted with correct message	**PASS**
TC-08	Reject product with zero price	Revert: 'Price must be greater than zero'	Transaction reverted with correct message	**PASS**
TC-09	Increment product counter correctly	productCounter = 2 after two products	getTotalProducts() returned 2	**PASS**
TC-10	Track products by business address	getBusinessProducts() returns [1, 2]	Array [1, 2] returned correctly	**PASS**
TC-11	Add a supply stage successfully	Stage stored with correct name, location, notes	Stage retrieved and all fields verified	**PASS**
TC-12	Reject stage from non-owner	Revert: 'Only the product owner can do this'	Transaction reverted with correct message	**PASS**
TC-13	Track multiple stages in order	getStageCount() = 3, stages in correct order	Three stages retrieved in correct order	**PASS**
TC-14	Emit StageAdded event	Event emitted with correct productId, stageName, location	Event verified with correct arguments	**PASS**
TC-15	Reject stage with empty stage name	Revert: 'Stage name cannot be empty'	Transaction reverted with correct message	**PASS**

TC-16	Allow buyer to purchase a product	ETH transferred to seller, purchase recorded	Seller balance increased, purchase confirmed	**PASS**
TC-17	Mark product unavailable after purchase	isAvailable = false after purchase	Product availability set to false	**PASS**
TC-18	Reject purchase with insufficient payment	Revert: 'Insufficient payment'	Transaction reverted with correct message	**PASS**
TC-19	Reject producer buying own product	Revert: 'Producer cannot buy their own product'	Transaction reverted with correct message	**PASS**
TC-20	Emit ProductPurchased event	Event with correct productId, buyer, seller, price	Event verified with all correct arguments	**PASS**
TC-21	Owner can toggle product availability	isAvailable = false after setProductAvailability(1, false)	Product availability toggled successfully	**PASS**
TC-22	Reject purchase of unavailable product	Revert: 'Product is not available for sale'	Transaction reverted with correct message	**PASS**
TC-23	Reject availability change from non-owner	Revert: 'Only the product owner can do this'	Transaction reverted with correct message	**PASS**

5.3 Terminal Output

The following output was produced by running npx hardhat test in the project directory:

```
WomenSupplyChain
  Business Registration
    ✔ TC-01: Should register a new business successfully (1647ms)
    ✔ TC-02: Should reject duplicate business registration (190ms)
    ✔ TC-03: Should reject empty business name (203ms)
    ✔ TC-04: Should reject empty location (52ms)
    ✔ TC-05: Should emit BusinessRegistered event (470ms)
  Product Registration
    ✔ TC-06: Should register a product successfully (1899ms)
    ✔ TC-07: Should reject product from unregistered business (45ms)
    ✔ TC-08: Should reject product with zero price (52ms)
    ✔ TC-09: Should increment product counter correctly
    ✔ TC-10: Should track products by business address (314ms)
  Supply Chain Tracking
    ✔ TC-11: Should add a supply stage successfully (449ms)
    ✔ TC-12: Should reject stage addition from non-owner (42ms)
    ✔ TC-13: Should track multiple stages in order (459ms)
    ✔ TC-14: Should emit StageAdded event (358ms)
    ✔ TC-15: Should reject stage with empty stage name
  Product Purchase
    ✔ TC-16: Should allow a buyer to purchase a product (969ms)
    ✔ TC-17: Should mark product as unavailable after purchase (168ms)
    ✔ TC-18: Should reject purchase with insufficient payment (42ms)
    ✔ TC-19: Should reject producer buying their own product (47ms)
    ✔ TC-20: Should emit ProductPurchased event (466ms)
  Product Availability
    ✔ TC-21: Should let owner toggle product availability (133ms)
    ✔ TC-22: Should reject purchase of unavailable product (40ms)
    ✔ TC-23: Should reject availability change from non-owner (65ms)

23 passing (52s)
```

6. Conclusion and Impact Assessment

6.1 Summary of Achievements

This project successfully designed, developed, tested, and deployed a blockchain-based supply chain transparency system tailored to the needs of women-led businesses in semi-urban Rwanda. The following deliverables were completed:

- A fully functional Solidity smart contract (WomenSupplyChain.sol) with 16 public functions, 5 events, and 3 access-control modifiers
- 23 automated test cases covering all major features, all passing on the Hardhat local network
- Successful deployment to the Ethereum Sepolia Testnet at address 0xAaB5646895d0F81dc45C0D6D8c3f9807db9F96D9
- A complete deployment guide reproducible on Ubuntu/WSL with Node.js and VS Code

6.2 Social Impact Potential

If deployed at scale in semi-urban Rwanda, this solution could:

- Help over 500,000 women SME owners in semi-urban Rwanda gain access to verifiable product histories
- Reduce counterfeit product competition by providing buyers with cryptographically verifiable product authenticity
- Enable women entrepreneurs to build on-chain business credit histories for microfinance access
- Eliminate middlemen who currently capture 30–50% of product value in traditional supply chains
- Connect women producers directly to NGO and international buyers who require supply chain transparency

6.3 Limitations and Future Work

The current implementation operates on a testnet with ETH, not real currency. Future development should consider:

- Integration with Rwandan mobile money (MTN MoMo) through an oracle bridge for fiat-adjacent payments
- A mobile-first front-end dApp optimized for low-bandwidth smartphone access in semi-urban areas
- Multi-language support in Kinyarwanda and French for the user interface
- Integration with Rwanda's national digital ID system (Irembo) for business verification
- Expansion to a Layer 2 solution (e.g., Polygon) to reduce gas costs for small transactions

Links:

Link to Github: https://github.com/Hirw23/women-supply-chain.git
Link to PowerPoint: https://canva.link/1u3tsb6efcjnr3h